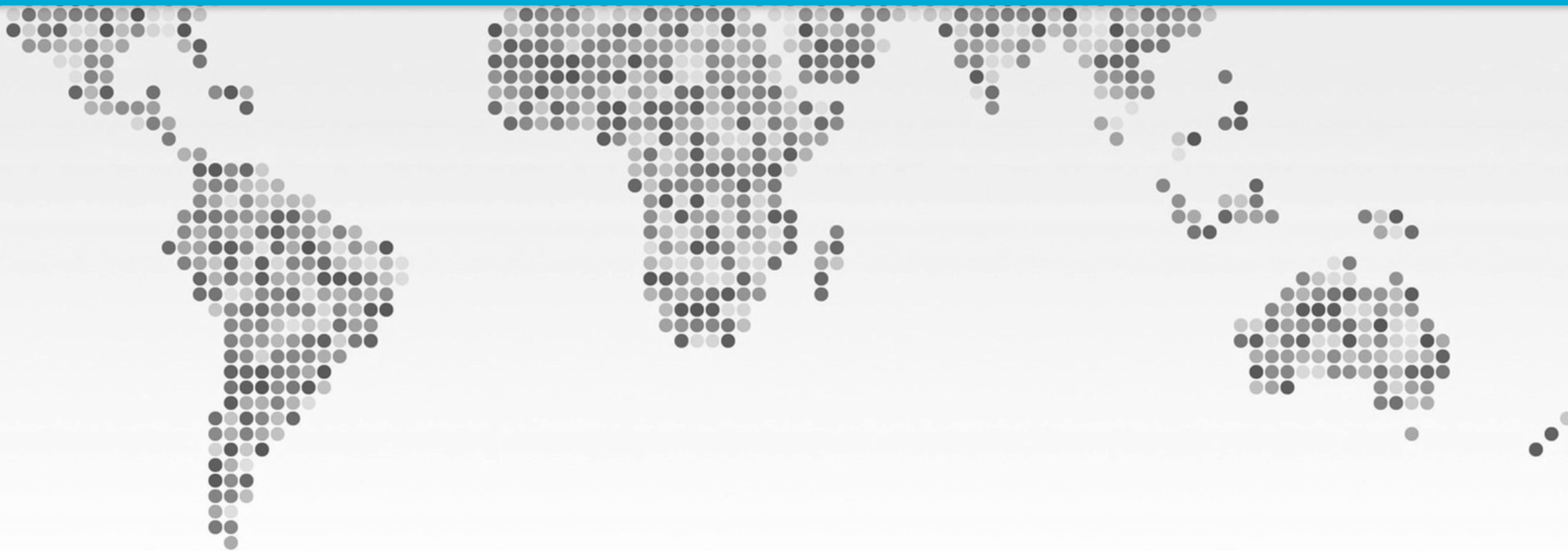


JUnit and Mockito

Software Dependability



How do we **verify** that **this code is correct**?

```
public class Calculator {  
    public int sum(int a, int b) {  
        return a + b;  
    }  
}
```

We **could write a test case** like the following one:

```
public class Main {  
    public static void main(String[] args) {  
        Calculator calc = new Calculator();  
        System.out.println(calc.sum(2, 10));  
        System.out.println(calc.sum(1, 0));  
        System.out.println(calc.sum(80, 13));  
    }  
}
```

What do we need to do?

Now, however, we can refine the skills we have acquired over these years.

Let us analyze the typical résumé of a software tester

WYMAN-KILBACK

02/2018 – present

San Francisco, CA // *Software Engineer, Test*

There are software testing frameworks!

- Passion for high quality software design and engineering
- Strong Unix skills, proficient with at least one shell and one high-level scripting language (bash, Python, Ruby)
- Familiarity with relational database design and implementation
- Familiarity with Jenkins, git, or other source control and CI/CD technologies

jUnit

Let's learn some terminology!

Test Case

A **set of expected inputs and outputs** that **exercise** a **component** of the **system** with the **aim** of **causing a failure** and **identifying faults**.

Test Suite

A **collection of tests** is called a test suite.

JUnit is an **open-source framework** to **write** and **run tests** in a **repeatable manner**.

N.B. Don't be **fooled** by the **name**!

JUnit can be **used** for **both unit tests** (which we will see today) and **other types of tests**!

Every framework used for writing test cases has different "**best practices**" that must be followed:

- Each **test case must** be **independent** of the others.
- It **must** be **able** to **identify** and **report errors** for each **test case**.
- It **must allow test cases** to be **written** and **executed simply**.

JUnit Design Goals

- **Easy** to **configure** and **integrate** into a **project**.
- **Allows** you to write **test cases** in a very **intuitive way**.
- **Allows** you to **reuse "portions"** of **other test cases**.

JUnit Configuration

One of the **simplest ways** to use **JUnit** is to **add** it as a **dependency** in the **POM.XML** file (for projects using the Maven build system) or in the **BUILD.GRADLE** file (for projects using the Gradle build system).

pom.xml

```
<dependency>
  <groupId>junit</groupId>
  <artifactId>junit</artifactId>
  <version>4.12</version>
  <scope>test</scope>
</dependency>
```

build.gradle

```
test {
    useJUnitPlatform()
}
```

```
dependencies {
    testImplementation 'org.junit.jupiter:junit-jupiter-api:5.8.1'
    testRuntimeOnly 'org.junit.jupiter:junit-jupiter-engine:5.8.1'
}
```

After configuring the project using the build system, a **"test"** directory will automatically appear where you can write your test cases.

What are Annotations?

Annotations

We define an **annotation** as a **string** that we **write above** a **method** or a **class**.

To define an **annotation** in **Java**, the annotation **must be preceded** by the "@" symbol.

Why use Annotations?

We **use annotations** to **allow** the **JVM** to **recognize** a **specific behavior** in a method/class.

JUnit Annotations

Annotation	Why is it used?
@Test	It indicates that the <u>method is a test case</u> .
@Before	It is used to execute a common piece of code <u>before</u> each test.

N.B. There are dozens and dozens of annotations; the ones listed are the most common!

@BeforeClass	It is used to execute a "piece of code" <u>before</u> the test suite begins.
@AfterClass	It is used to execute a "piece of code" <u>after</u> the test suite begins.
@RepeatedTest(n)	It is used when we want to repeat a specific test case a certain number of times.

When are these annotations used? - Practical examples

```
@Test
public void testNameMethodToTest() {
    //instructions
}
```

```
@RepeatedTest(value=10)
public void test_NameMethod() {
    //instructions
}
```

```
@After
public void teardown() {
    list = null()
}
```

```
private List<String> list;

@Before
public void init() {
    list = new ArrayList<>(Arrays.asList("a1", "b1"));
}
```

```
@BeforeClass
public static void setup() {
    File file = new File("C:\\test\\test.txt");
}
```

```
@AfterClass
public static void tearDown() {
    file.close();
}
```

What are Assertions?

Assertions

Assertions are used to **verify whether a test case has succeeded or failed**.

Assert

Among the many assertions that JUnit offers, we can mention the following:

A test case should test only one thing!
Therefore, you should have a single assert statement for each test case!

	<code>assertArrayEqual("message", a, b)</code>	Compare if two arrays are equal.
	<code>assertEqual("message, a, b)</code>	Compare if two objects are equal.

A test case, in order to be considered a test case, MUST have an assertion!

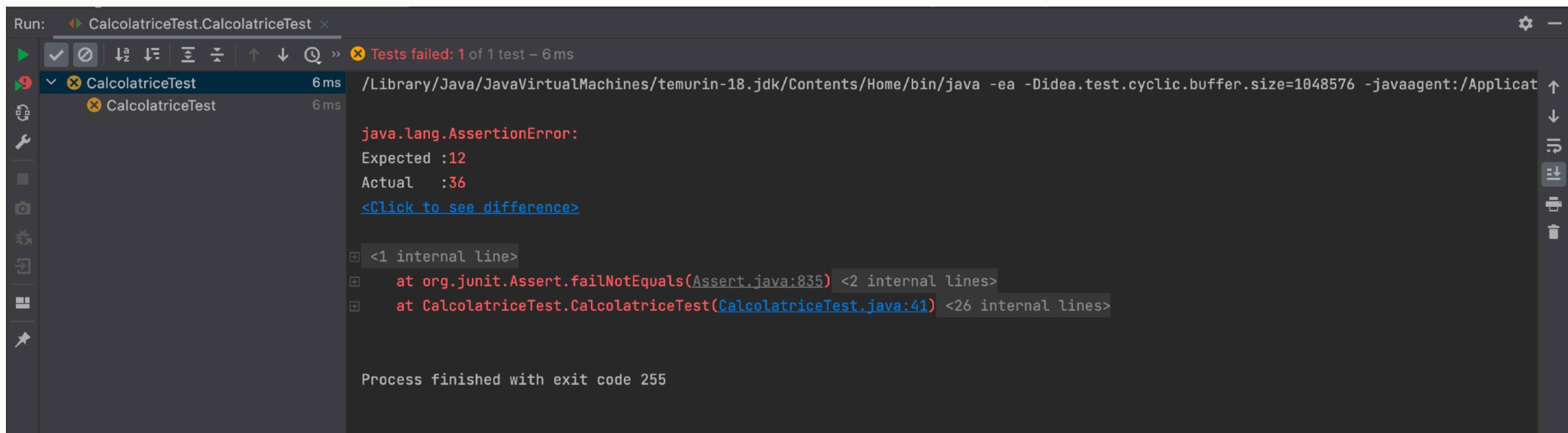
	<code>assertFalse(a)</code>	Check if the status of that object is false.
--	-----------------------------	--

Assert

Is this test case correct?

```
@Test
public void calculatorTest() throws Exception {
    Calculator calc = new Calculator();
    assertEquals(12, calc.multiply(6, 6));
    assertEquals(1, calc.subtract(10, 9));
    assertEquals(93, calc.divide(930, 3));
}
```

This test case tests multiple functionalities!



```
Run: CalcolatriceTest.CalcolatriceTest x
Tests failed: 1 of 1 test - 6 ms
CalcolatriceTest 6 ms
CalcolatriceTest 6 ms
/Library/Java/JavaVirtualMachines/temurin-18.jdk/Contents/Home/bin/java -ea -Didea.test.cyclic.buffer.size=1048576 -javaagent:/Applicat
java.lang.AssertionError:
Expected :12
Actual   :36
<Click to see difference>
<1 internal line>
at org.junit.Assert.failNotEquals(Assert.java:835) <2 internal lines>
at CalcolatriceTest.CalcolatriceTest(CalcolatriceTest.java:41) <26 internal lines>
Process finished with exit code 255
```

It is impossible to determine which component is defective in the event of a failure!

Assert

And this?

```
@Test
public void CalculatorTest() throws Exception {
    Calculator calc = new Calculator();
    System.out.println(12 == calc.multiply(6, 6));
}
```

The Assert is missing!

The following test case verifies the correctness of a method implementing a sum:

```
public class CalculatorTest {  
    Calculator calculator;  
  
    @Before  
    public final void setUp() {  
        calculator = new Calculator();  
    }  
  
    @Test  
    public void testSum() {  
        assertTrue("correct sum", calculator.sum(11, 1) == 12);  
    }  
}
```

If we want to **run the same test case** with **different inputs**, we can use the **@ParameterizedTest** and **@MethodSource()** annotations.

```
public class CalculatorTest {  
    public static int[][] dataSetForAdd() {  
        return new int[][] { {1, 2, 3}, {2, 4, 6}, {121, 4, 125} };  
    }  
  
    @ParameterizedTest  
    @MethodSource(value = "dataSetForAdd")  
    void testCalculatorAddMethod(int[] dataSetForAdd) {  
        Calculator calc = new Calculator();  
        int m1 = dataSetForAdd[0];  
        int m2 = dataSetForAdd[1];  
        int expected = dataSetForAdd[2];  
        assertEquals(expected, calc.add(m1, m2));  
    }  
}
```



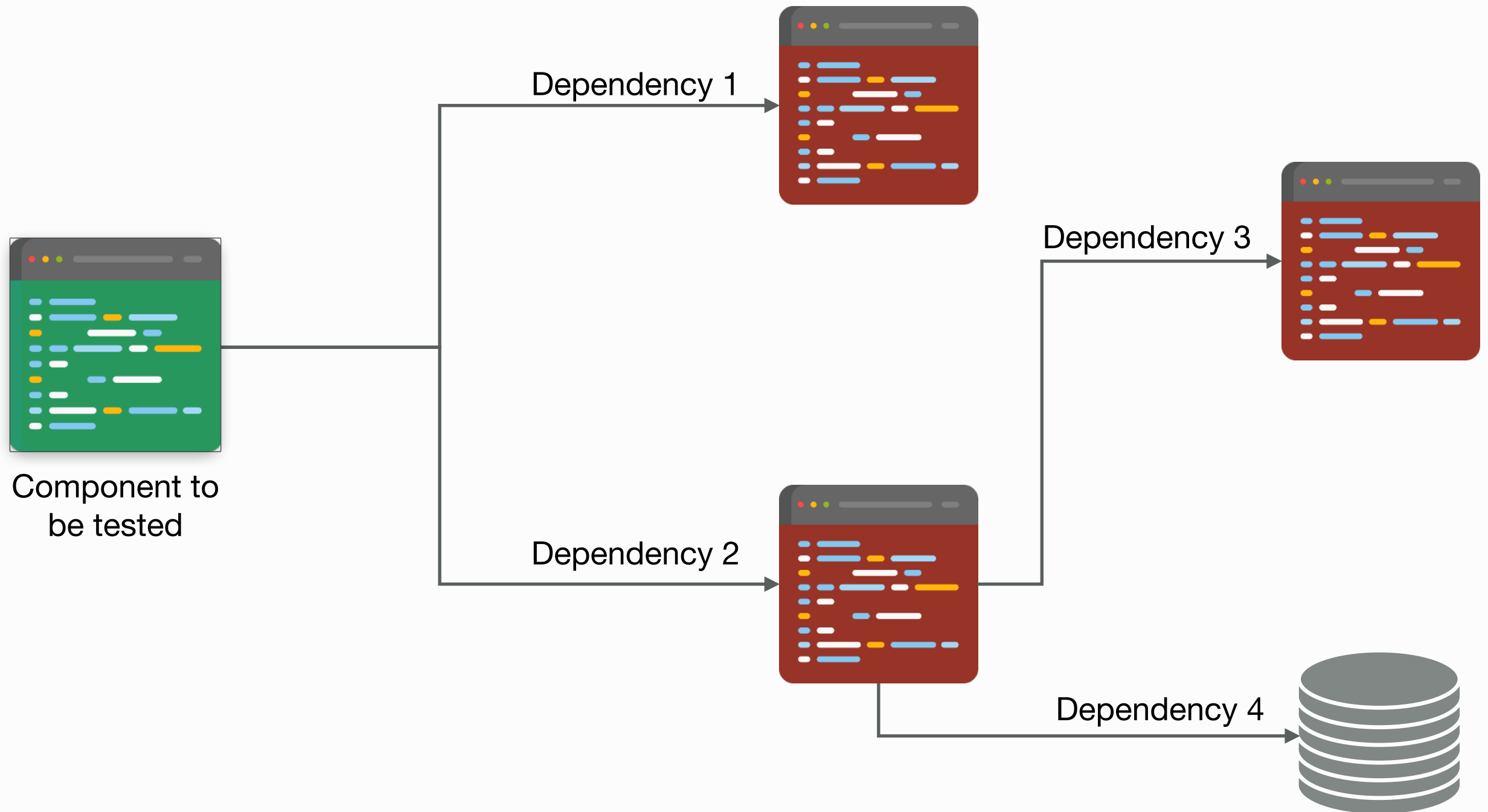
We can create Mocks!

What if we needed to test something more 'complex'?

So far, **we have seen examples** where the **component under test had no dependencies** on other components; however, **this may not always be the case**.

Software is often complex and **requires many elements** to function properly

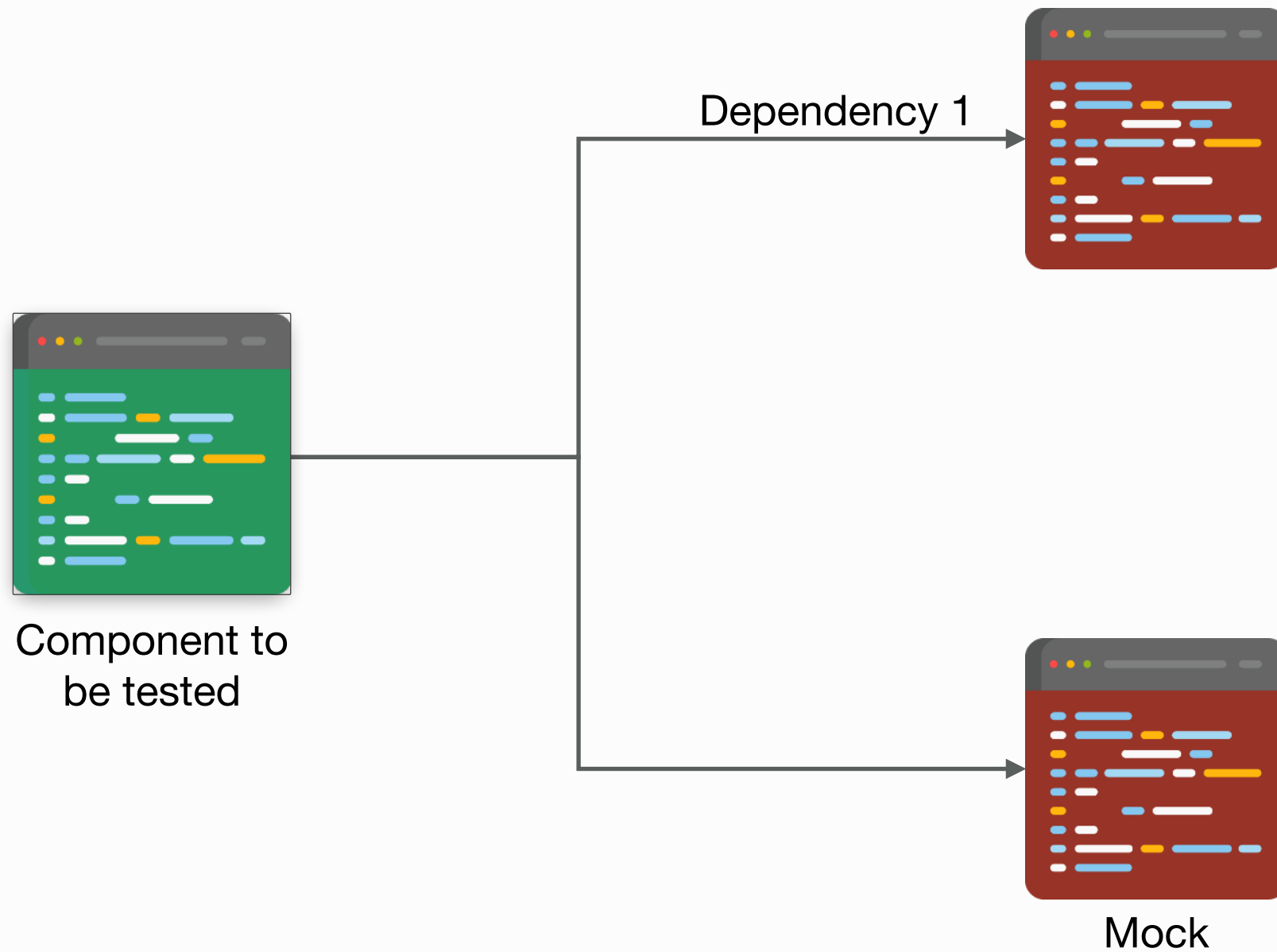
Mock...How does it work?



The component to be tested has a series of dependencies, some of which might be complex to instantiate.

Mock

Mock...How does it work?



We can create a mock, i.e., an object that simulates the behavior of a complex component.

Why Mock?

Creating a Mock object can be useful in the following cases:

- Working in parallel
- Avoiding changes to the production code to make a certain dependency return a specific value
- Writing and running unit tests quickly
- Improving the design of the source code
- Simulating database access
- Etc...

But why not just create a stub?

Let us analyze the main differences between a Mock and a Stub.

VS	Mock	Stub
Behavior	Dynamic Behavior	Predefined Behavior
Source Code	Use of Polimorfism	Source code replacement
Implementation	All methods are implemented	Only the methods used by the test are implemented



Mockito...How does it work?

Mockito works according to the GWT principle, that is:

- **Given:** Describes the state or environment *before* the mock begins (a sort of pre-condition)
- **When:** Describes the trigger that must occur to cause a response
- **Then:** Describes the changes that must be made

Among the various methods that Mockito provides, the fundamental ones are the following:

- `mock(ClassToMock)`
- `when(Method call)`
- `thenReturn(value)`

Example

Let us consider the following test case. The **Given**, **When**, and **Then** operations are the following:

```
@test
public void testCalculator(){
    //Given
    Calculator calc = new Calculator();
    SecondDependency mockDep = new Mock(SecondDependency.class);
    when(mockDep.method()).thenReturn("ok");

    //When
    int a = calc.sum(11,1);

    //Then
    assertEquals(12,a);
}
```

Example

We can use the following mock

```
@test
public void testCalculator(){
    //Given
    Calculator calc = new Calculator();
    SecondDependency mockDep = new Mock(SecondDependency.class);
    when(mockDep.method()).thenReturn("ok");

    //When
    int a = calc.sum(11,1);

    //Then
    assertEquals(12,a);
}
```

We simulated a call to the 'SecondDependency' method and assumed that it returns 'ok'.

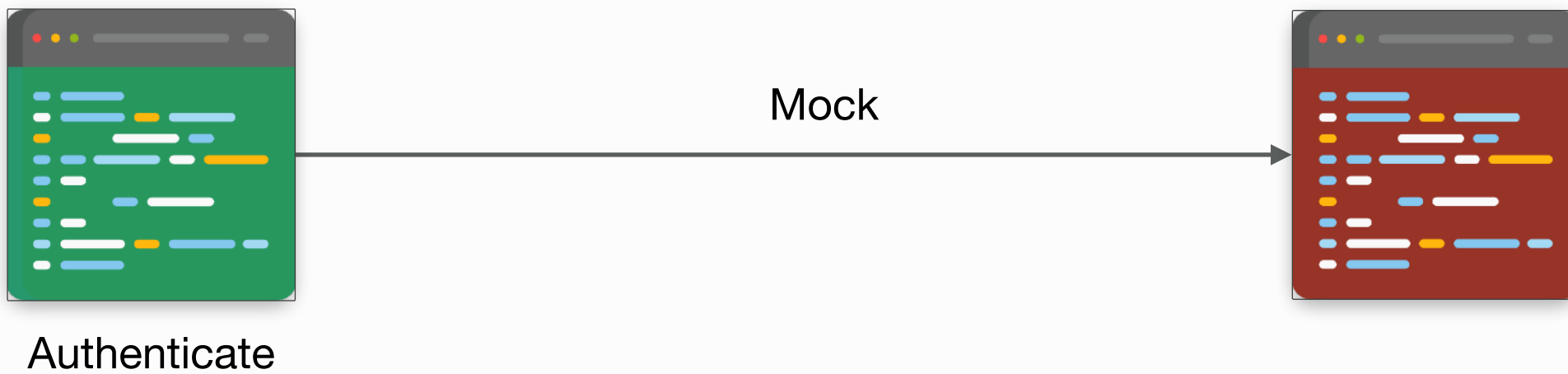
Example

Let us **suppose we want to test** the **login** method, but it **requires database access**.

```
public interface AuthenticatorInterface {  
    public boolean authenticateUser(String username, String password) throws EmptyCredentialsException;  
}
```

```
public class AuthenticatorApplication {  
    private AuthenticatorInterface authenticator;  
  
    public AuthenticatorApplication(AuthenticatorInterface authenticator) {  
        this.authenticator = authenticator;  
    }  
  
    public boolean authenticate(String username, String password) throws NotAuthenticatedException {  
        boolean authenticated;  
        authenticated = this.authenticator.authenticateUser(username, password);  
        return authenticated;  
    }  
}
```

Mock... How does it work?



Create a mock, i.e., an object that simulates the behavior of a complex component.

Example

We want **write** the **following test case**:

```
public class AuthenticatorApplicationTest {  
  
    @Test  
    public void testAuthenticate() throws EmptyCredentialsException {  
        AuthenticatorInterface authenticatorMock;  
        AuthenticatorApplication authenticator;  
        String username = "username";  
        String password = "noncorrectpassword";  
  
        authenticatorMock = Mockito.mock(AuthenticatorInterface.class);  
        authenticator = new AuthenticatorApplication(authenticatorMock);  
  
        when(authenticatorMock.authenticateUser(username, password)).thenReturn(false);  
  
        boolean actual = authenticator.authenticate(username, password);  
        assertFalse(actual);  
    }  
}
```

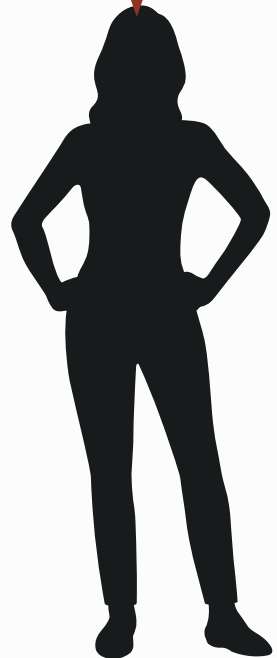
We created a mock of the **AuthenticatorInterface** and simulated its behavior by instructing **Mockito** to return **false** when the 'authenticate' method is called.

We have tested a method by simulating a database interaction, turning it into a mock method.

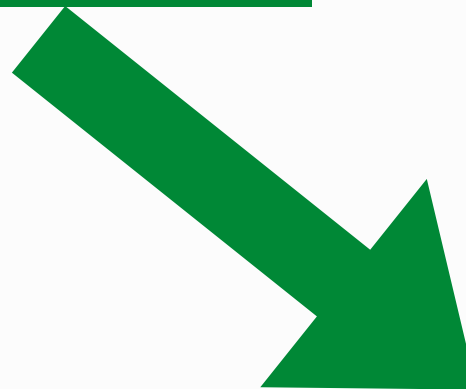
We tested the login procedure when it fails. Although we did not implement the credential verification method in the database, we could still test its behavior by testing the login procedure directly.

Example

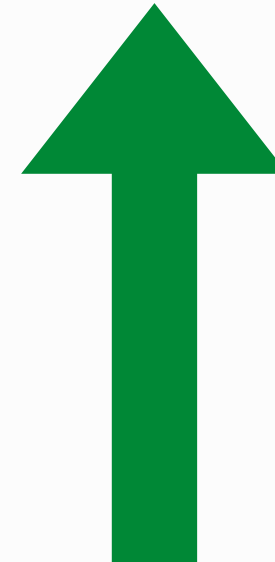
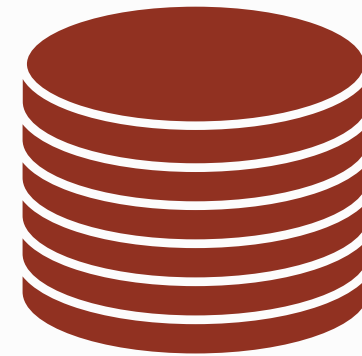
We want to filter all the invoices with a value lower than 100. The invoices are stored in a database.



InvoiceDAOTest



InvoiceDAO



Example

```
public List<Invoice> filter() {  
    InvoiceDao dao = new InvoiceDao();  
    List<Invoice> all= dao.all();  
    List<> filtered = new ArrayList<>();  
  
    for(Invoice inv : all) {  
        if(inv.getValue() < 100.0)  
            filtered.add(inv);  
    }  
  
    return filtered;  
}
```

Example

```
public List<Invoice> filter() {  
    InvoiceDao dao = new InvoiceDao();  
    List<Invoice> all = dao.findAll();  
    List<> filtered = new ArrayList<>();  
  
    for (Invoice inv : all) {  
        if (inv.getValue() < 100.0)  
            filtered.add(inv);  
    }  
  
    return filtered;  
}
```

One might think that these are the two most complex parts to test.



Example

```
public List<Invoice> filter() {  
    InvoiceDao dao = new InvoiceDao();  
    List<Invoice> all = dao.all();  
    List<> filtered = new ArrayList<>();  
  
    for (Invoice inv : all) {  
        if (inv.getValue() < 100.0) {  
            filtered.add(inv);  
        }  
    }  
  
    return filtered;  
}
```

Instead, this is the most complex part!

Example

Let's take a look at the test case.

```
@Test void filterInvoices() {  
    InvoiceDao dao = new InvoiceDao();  
    Invoice i1 = new Invoice("M", 20.0);  
    Invoice i2 = new Invoice("A", 300.0);  
    dao.save(i1); dao.save(i2);  
  
    InvoiceFilter f = new InvoiceFilter();  
    List<Invoice> result = f.filter();  
  
    assertEquals(1, result.size());  
    assertEquals(i1, result.get(0));  
    dao.close();  
}
```

Example

Let us take a pause... We agreed that **each test case should have only one assert...**

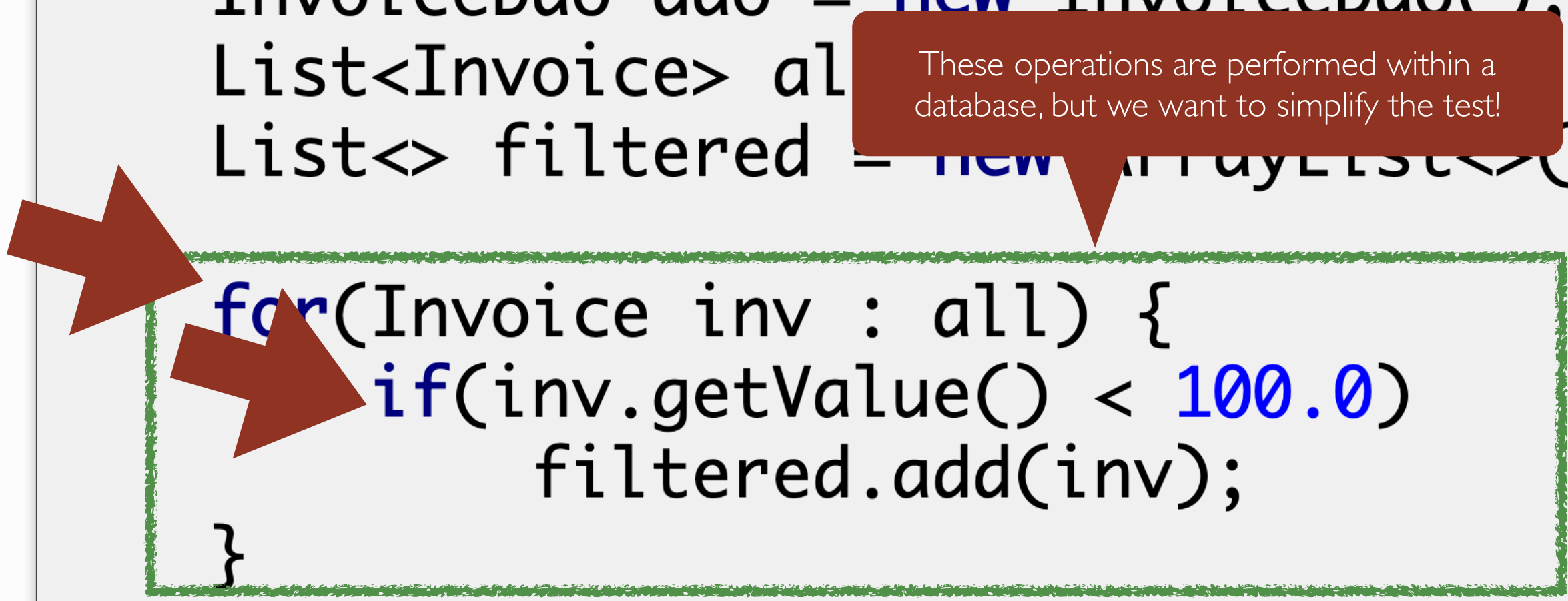
```
assertEquals(1, result.size());  
assertEquals(i1, result.get(0));
```

This is true, but **it is allowed not to replicate the test code** almost line by line.
Furthermore, **the test still verifies a single functionality.**

Example

```
public List<Invoice> filter() {  
    InvoiceDao dao = new InvoiceDao();  
    List<Invoice> all = dao.findAll();  
    List<> filtered = new ArrayList<>();  
  
    for (Invoice inv : all) {  
        if (inv.getValue() < 100.0)  
            filtered.add(inv);  
    }  
  
    return filtered;  
}
```

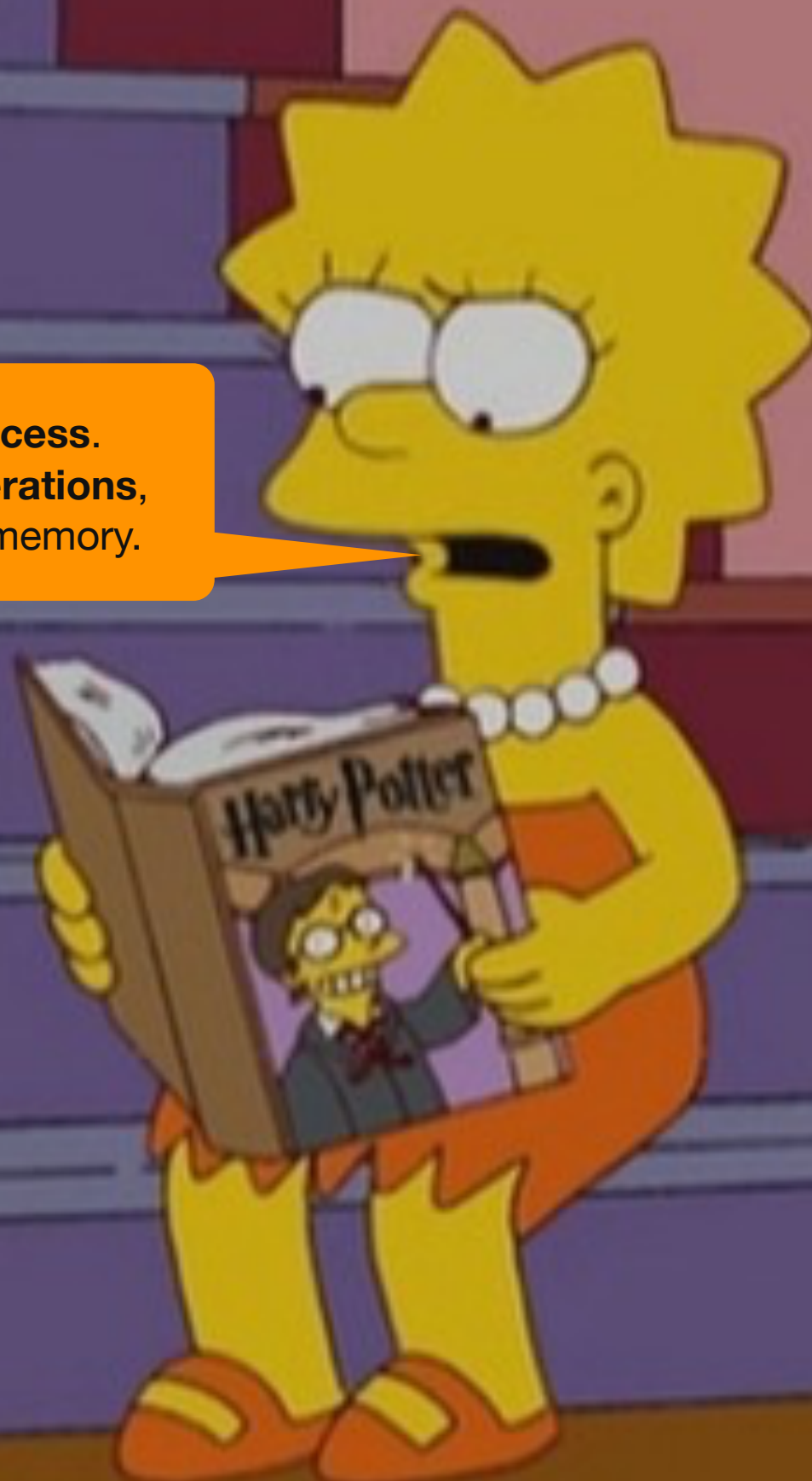
These operations are performed within a database, but we want to simplify the test!

A diagram illustrating the flow of the code. A green rectangular box highlights the loop section: `for (Invoice inv : all) { if (inv.getValue() < 100.0) filtered.add(inv); }`. Two red arrows point from the left towards the `for` and `if` statements. A red speech bubble points to the `new InvoiceDao()` line, containing the text: "These operations are performed within a database, but we want to simplify the test!".



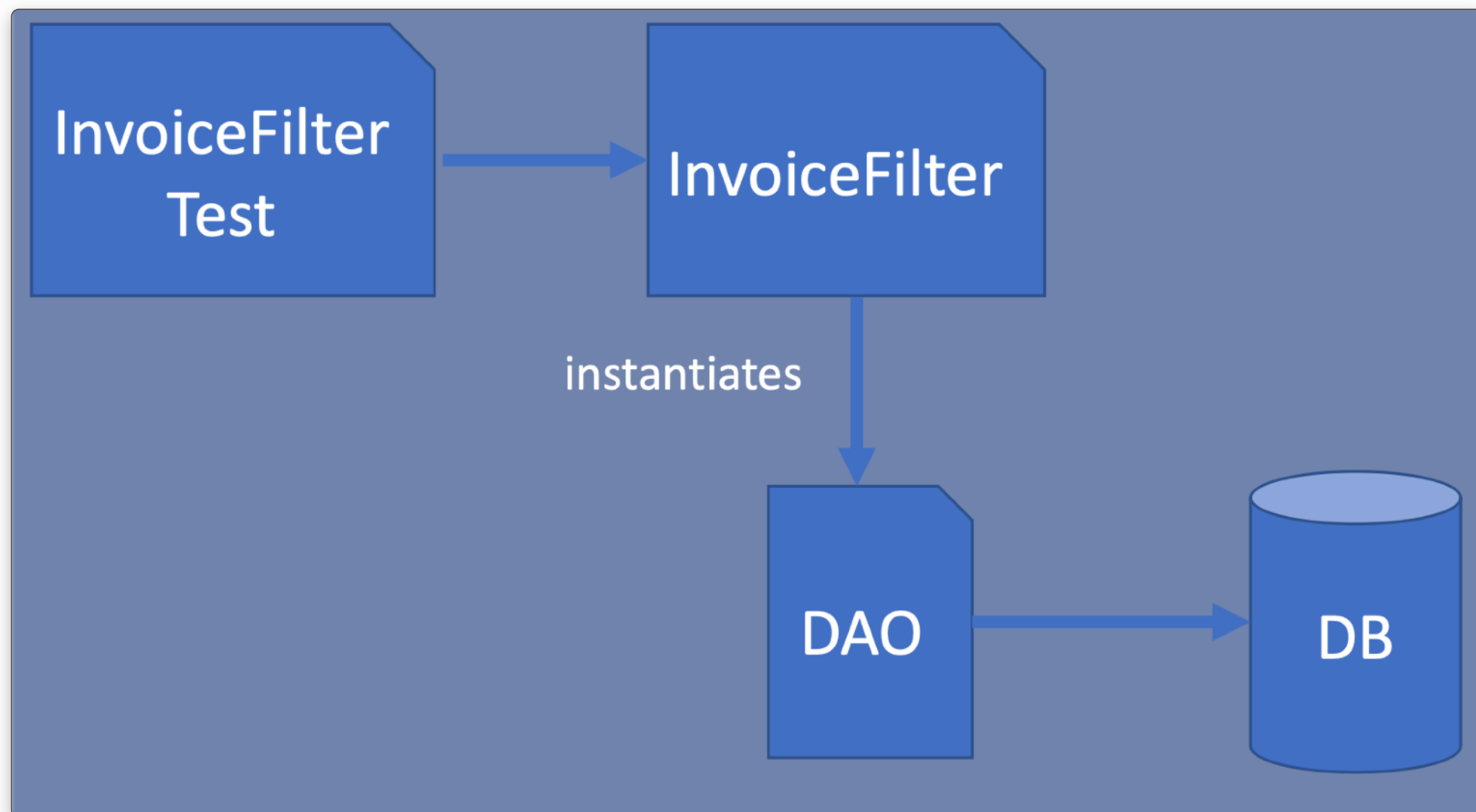
So how do we do it?

Let's create a **mock object** for database access.
This simulation will perform **simple fake operations**,
such as returning a list of invoices stored in memory.



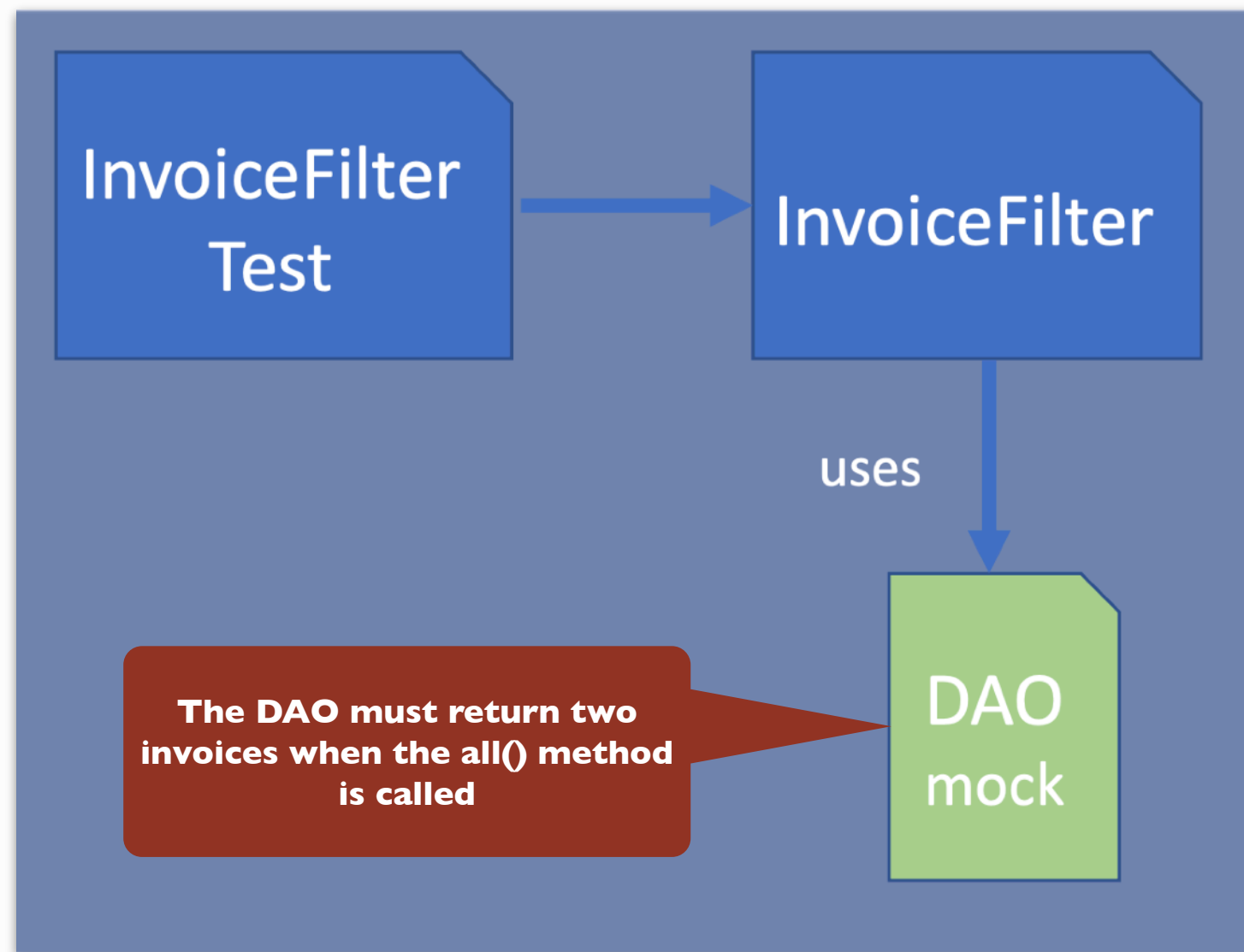
Example

Let's create **a mock object for database access**. This simulation will perform **simple fake operations**, such as returning a list of invoices stored in memory.



Example

Let's create **a mock object for database access**. This simulation will perform **simple fake operations**, such as returning a list of invoices stored in memory.



Example

```
@Test void filterInvoices() {  
    InvoiceDao dao = new InvoiceDao();  
    Invoice i1 = new Invoice("M", 20.0);  
    Invoice i2 = new Invoice("A", 300.0);  
    dao.save(i1); dao.save(i2);  
  
    InvoiceFilter f = new InvoiceFilter();  
    List<Invoice> result = f.filter();  
  
    assertEquals(1, result.size());  
    assertEquals(i1, result.get(0));  
    dao.close();  
}
```

Example

```
@Test void filterInvoices() {  
    Invoice i1 = new Invoice("M", 20.0);  
    Invoice i2 = new Invoice("A", 300.0);  
    InvoiceDao dao =  
        Mockito.mock(InvoiceDao.class);  
  
    List<Invoice> list = Arrays.asList(i1,i2);  
    Mockito.when(dao.all()).thenReturn(list);  
  
    InvoiceFilter f = new InvoiceFilter();  
    List<Invoice> result = f.filter();  
    // assertions  
}
```

Limits

Limiti	
Enum	
Metodi Statici	
Metodi Privati	
HashCode	
Equals	



N.B. The **framework** is **continuously evolving**!

If you need to **test components** with these **characteristics**, you can use **PowerMock**

How to configure it?

```
<dependency>
  <groupId>org.mockito</groupId>
  <artifactId>mockito-core</artifactId>
  <version>4.9.0</version>
  <scope>test</scope>
</dependency>
```

POM.XML

```
testImplementation group: 'org.mockito', name: 'mockito-core', version: '4.9.0'
```

BUILD.GRADLE

JUnit and Mockito

Software Dependability

